

Relazione Progetto 2 MdCS

Andrea Coldani (894684) Fabio Colonetti (899689)
Francesco Fusi (899741)

Giugno 2026

1 Introduzione

L'elaborato descrive l'analisi e lo sviluppo di un sistema software per la compressione di immagini in toni di grigio. Gli obiettivi del progetto si articolano in due fasi principali:

- **Analisi prestazionale:** dedicata all'implementazione in Python dell'algoritmo DCT2 e confronto approfondito con la routine FFT della libreria `scipy`.
- **Compressione JPEG:** sviluppo di un sistema di compressione d'immagine in scala di grigio basato sullo standard JPEG (senza l'ausilio della matrice di quantizzazione).

2 Struttura della libreria

Il software è stato sviluppato con il linguaggio Python e strutturato in modo modulare per garantire separazione tra la logica di calcolo matematico, l'interfaccia utente, i test di validazione. Di seguito viene descritto il ruolo di ciascuna cartella e file presenti nella radice del progetto:

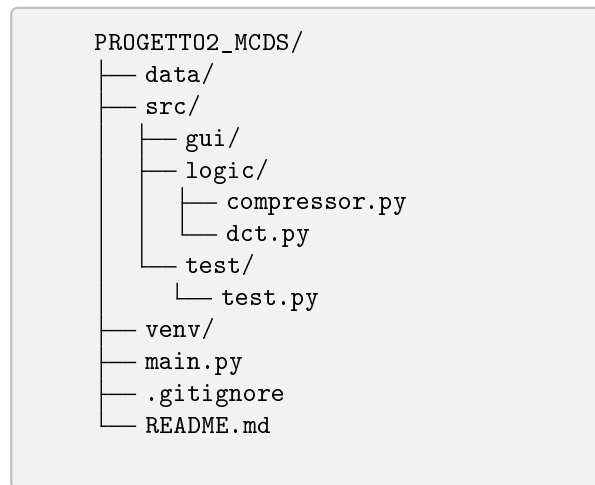


Figura 1: Struttura delle directory del progetto

- **data/**: Cartella contenente le immagini in formato bitmap (**.bmp**) in toni di grigio, utilizzate come input per la compressione.
- **src/** (Source): Contiene l'intero codice sorgente dell'applicazione, suddiviso nei seguenti sotto-moduli specializzati:
 - **gui/**: Contiene il codice che implementa l'interfaccia grafica che consente all'utente di navigare nel filesystem per selezionare l'immagine e inserire i parametri numerici di input (F e d).
 - **logic/**: Contiene il nucleo computazionale dell'applicativo:
 - * **dct.py**: Contiene l'implementazione sequenziale della DCT/-DCT2 definita "fatta in casa" ($O(N^3)$) e le funzioni di interfaccia con la libreria scientifica per la versione "fast" ($O(N^2 \log N)$), necessarie al confronto dei tempi di esecuzione.
 - * **compressor.py**: Contiene l'algoritmo di compressione JPEG-like, occupandosi della segmentazione dell'immagine in macro-blocchi $F \times F$, del filtraggio delle frequenze oltre la diagonale d .
 - **test/test.py**: Contiene Script di Landmark-Testing utilizzato per verificare la correttezza numerica e lo scaling delle funzioni implementate, confrontando i risultati della DCT monodimensionale e bidimensionale con la matrice di test 8×8 fornita nelle specifiche.
- **main.py**: Il punto di ingresso (*entry-point*) principale dell'applicazione che inizializza l'interfaccia utente e coordina il flusso dei dati tra i vari moduli.

- `venv/`, `.gitignore`, `README.md`: File e directory di configurazione per la gestione dell'ambiente virtuale isolato, l'esclusione di file ridondanti dal controllo di versione e la stesura delle istruzioni rapide per l'installazione e l'esecuzione del programma.

3 parte 1

3.1 Analisi delle Prestazioni della DCT2 e confronto con FFT

L'algoritmo implementato da noi, applica la definizione classica della trasformata sfruttando la proprietà di separabilità. La DCT2 viene quindi calcolata come due trasformate monodimensionali in cascata (prima per righe e poi per colonne) tramite prodotti matrice-matrice del tipo $c = T \cdot f \cdot T^T$. Ciascun prodotto richiede l'esecuzione di cicli annidati su strutture dati di dimensione N , portando il costo computazionale teorico a un ordine di complessità pari a $\mathcal{O}(N^3)$.

Al contrario, la routine della libreria ottimizzata si appoggia ad algoritmi di tipo Fast correlati alla Trasformata Rapida di Fourier (FFT). Invece di eseguire il prodotto matriciale diretto, la FFT riduce drasticamente il numero di operazioni ridondanti. Sfruttando la simmetria e la ricorsività (secondo una logica *divide et impera*), il problema di dimensione N viene suddiviso in sottoproblemi più piccoli. Questo approccio abbatta la complessità computazionale del calcolo bidimensionale a:

$$\mathcal{O}(N^2 \log_2 N) \quad (1)$$

dove N^2 rappresenta il numero totale di pixel dell'immagine, mentre il fattore logaritmico descrive l'efficienza della scomposizione in frequenza.

3.1.1 Dati Sperimentali Raccolti

La validazione sperimentale è stata condotta su matrici quadrate di dimensione N crescente [128, 256, 512], mediando i tempi su 100 run indipendenti per abbattere il rumore di fondo.

I risultati, riassunti nella Tabelle 1 e il grafico in Figura 2, confermano sperimentalmente i modelli di complessità teorica precedentemente descritti.

N	Tempo Medio Manuale $\mathcal{O}(N^3)$ [s]	Rapporto $T(N)/T(N/2)$
128	1.767253×10^{-1}	—
256	1.771358×10^0	10.02
512	1.404507×10^1	7.93

Tabella 1: Tempi medi di esecuzione nostra implementazione al variare di N .

L'andamento dell'algoritmo implementato da noi mostra una crescita dei tempi $\mathcal{O}(N^3)$. Quando la dimensione della matrice raddoppia, passando da

$N = 256$ a $N = 512$, il tempo teorico di esecuzione dovrebbe aumentare di un fattore pari a $2^3 = 8$. Dai dati sperimentali reali si osserva un incremento effettivo estremamente coerente con la teoria, pari a 7.93, confermando quanto detto in precedenza.

Il picco registrato nel passaggio precedente da $N = 128$ a $N = 256$, pari a un rapporto di 10.02 è invece dovuto a colli di bottiglia hardware, in quanto matrici 128×128 possono essere salvate in cache, mentre matrici 256×256 saturano la memoria cache della CPU (*cache miss*) e dunque passano alla memoria RAM, più lenta, che altera dunque il rapporto.

N	Tempo Medio Libreria $\mathcal{O}(N^2 \log_2 N)$ [s]	Rapporto $T(N)/T(N/2)$
128	4.061909×10^{-4}	—
256	1.367665×10^{-3}	3.37
512	5.212104×10^{-3}	3.81

Tabella 2: Tempi medi di esecuzione per l'algoritmo di libreria SciPy al variare di N .

Per quanto riguarda l'algoritmo di libreria, la complessità attesa segue $\mathcal{O}(N^2 \log_2 N)$. In questo caso al raddoppiare della dimensione della matrice il rapporto di crescita teorico non è costante ma decresce al crescere di N . Dunque i rapporti teorici attesi sono pari a:

$$\text{Target Teorico } (128 \rightarrow 256) : \frac{256^2 \cdot \log_2(256)}{128^2 \cdot \log_2(128)} = 4 \cdot \frac{8}{7} \approx 4.57 \quad (2)$$

$$\text{Target Teorico } (256 \rightarrow 512) : \frac{512^2 \cdot \log_2(512)}{256^2 \cdot \log_2(256)} = 4 \cdot \frac{9}{8} = 4.50 \quad (3)$$

Calcolando sperimentalmente, i rapporti sui tempi medi emerge uno scostamento rispetto al modello teorico puro:

$$\text{Rapporto Reale } (128 \rightarrow 256) : \frac{1.367665 \times 10^{-3}}{4.061909 \times 10^{-4}} \approx 3.37 \quad (4)$$

$$\text{Rapporto Reale } (256 \rightarrow 512) : \frac{5.212104 \times 10^{-3}}{1.367665 \times 10^{-3}} \approx 3.81 \quad (5)$$

La differenza tra i valori teorici e quelli sperimentali si spiega analizzando le componenti hardware e software del sistema. In entrambi i casi notiamo che il rapporto reale è inferiore a quello teorico. Questo evidenzia l'efficacia delle ottimizzazioni a basso livello della libreria SciPy (scritta in C/Fortran). A queste dimensioni intermedie, l'algoritmo sfrutta appieno la vettorizzazione dei registri della CPU e il parallelismo hardware, abbattendo i tempi di calcolo oltre la semplice previsione matematica.

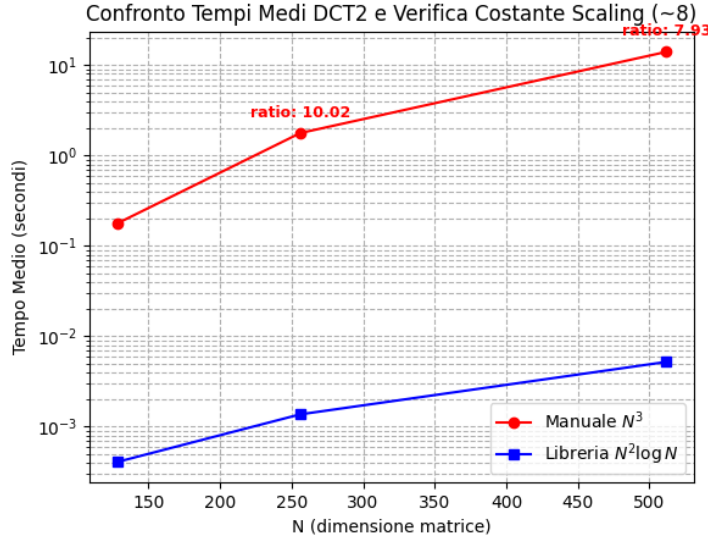


Figura 2: Andamento dei tempi medi di esecuzione al variare della dimensione N .

In conclusione possiamo dire che al crescere di N , il divario prestazionale tra i due approcci diventa significativo. Alla dimensione massima testata di $N = 512$, la routine ottimizzata conclude il calcolo in appena 5.21 millisecondi (5.212104×10^{-3} s), risultando circa 2695 volte più rapida dell'algoritmo manuale da noi implementato, il quale richiede invece ben 14.04 secondi (1.404507×10^1 s).

4 parte 2

Fissato un $F = 8$ fare i confronti ma nella

fissato $F=8$ fare i confronti aumentando d e diminuendo d (prenderei come valore di riferimento il 25) Fissato F e d fare un confronto tra MSE e visivo tra un'immagine naturale e un a scacchiera Qualche cosa di qualitativo aumentando la dimensione di F

Per valutare il comportamento e l'efficienza dell'algoritmo implementato, sono stati adottati tre diversi approcci di valutazione: come metrica di valutazione quantitativa è stato utilizzato l'Errore Quadratico Medio (MSE) e come metrica di valutazione qualitativa un'analisi visiva delle immagini risultanti.

4.0.1 Variazione del parametro di taglio d (Fissato $F = 8$)

Come prima prova è stata fissata la dimensione del blocco a $F = 8$, variando il parametro d che rappresenta il numero dei coefficienti della DCT salvati.

Prendendo, 80x80.bmp come immagine e assumendo di salvare il 25% delle frequenze ($d \approx 5$) si ottiene un MSE pari a 30.55.

- **Salvando il 50% delle frequenze** ($d \approx 8$) l'MSE decresce assumendo il valore di 8.89. Dal punto di vista qualitativo, l'immagine risulta praticamente indistinguibile dall'originale. Il salvataggio dei coefficienti di ordine superiore introduce nel sistema le alte frequenze spaziali, necessarie per la ricostruzione accurata dei dettagli fini e dei contorni netti.
- **Salvando il 5% delle frequenze** ($d \approx 2$) l'MSE aumenta a 58.42. Questo porta ai due artefatti tipici della compressione:
 1. *Effetto blocco*: le discontinuità ai confini delle sottomatrici 8×8 diventano marcate.
 2. *Sfocatura*: la rimozione quasi totale delle alte frequenze cancella il gradino bianco nero, rendendo sfocato il passaggio dal bianco al nero.

4.0.2 Confronto tra Immagine Naturale e Scacchiera (Fissati F e d)

Mantenendo fissi i parametri $F = 8$ (dimensione di blocco) e $d = 5$ (soglia di taglio), l'algoritmo viene applicato a due immagini in input con caratteristiche spettrali opposte: un'immagine fotografica naturale *cattedral.bmp* e un'immagine sintetica geometrica a scacchiera *80x80.bmp*. Da un'analisi qualitativa emerge che:

- L'immagine naturale presenta un valore di MSE pari a 2.20 e una stabilità visiva ottimale. Le immagini reali sono segnali a bassa variazione locale, caratterizzati da forti correlazioni spaziali.
- L'immagine a scacchiera invece evidenzia un MSE pari a 30.55, valore elevato e una sfocatura visiva. La transizione istantanea tra bianco e nero puro richiede uno spettro elevato di alte frequenze per essere ricostruita.

Questo confronto dimostra che l'efficienza della trasformata DCT non è assoluta, ma strettamente vincolata alla natura del segnale informativo. Essa si rivela una base ottimale per l'approssimazione di segnali continui e morbidi (immagini naturali), mentre degrada rapidamente se applicata a grafiche geometriche ad alto contrasto locale.

4.0.3 Analisi Qualitativa all'Aumentare della Dimensione del Blocco F

In questa analisi si è valutato il comportamento della ricostruzione modificando il parametro F a dimensioni superiori ($F = 8, 16, 24, 32$) e mantenendo inalterata la proporzione di taglio al 25%.

Immagine cathedral.bmp

L'analisi dei dati quantitativi (Tabella 3) e delle ricostruzioni visive permette di evidenziare alcune differenze fondamentali del comportamento della DCT al variare del blocco F .

Dal punto di vista percettivo, la qualità dell'immagine rimane sorprendentemente stabile su tutte le configurazioni, senza perdite visibili di dettaglio. Tuttavia, l'indicatore numerico dell'errore (MSE) mostra un andamento chiaro: staziona intorno a 2.20 per $F = 8$ e $F = 16$, per poi subire un aumento a 16.78 ($F = 24$) e 19.49 ($F = 32$).

F	d	Percentuale	MSE
8	5	23.44%	2.20
16	11	25.78%	2.16
24	17	26.56%	16.78
32	22	24.71%	19.49

Tabella 3: MSE al variare di F , mantenendo costante il tasso di taglio al 25%.

Avendo escluso effetti di bordo tramite il taglio diretto della matrice all'altezza dell'ultimo blocco utile, questa crescita dell'MSE è legata esclusivamente alla natura spettrale della trasformata su aree estese. All'aumentare di F , il blocco racchiude al suo interno una quantità nettamente maggiore di dettagli, contorni e variazioni locali. Mantenendo la diagonale costante al 25%, la DCT si trova a dover approssimare zone molto più complesse ed eterogenee con lo stesso set proporzionale di basse frequenze. Visivamente questo effetto si nota solo esaminando i margini laterali dei singoli macro-blocchi, dove il troncamento netto delle frequenze più alte genera una lievissima perdita di coerenza spaziale inter-blocco. È proprio questa minima dispersione energetica su matrici grandi a far salire numericamente l'MSE, nonostante il nucleo centrale della foto preservi una fedeltà visiva ottimale.

Immagine 160x160.bmp

L'applicazione dell'algoritmo di compressione al pattern geometrico della scacchiera mostra un andamento del MSE più elevato rispetto all'immagine naturale e fortemente oscillante (Tabella 4): l'errore staziona intorno a 31 per $F = 8$ e $F = 16$, aumentando poi a 164.92 per $F = 24$ e diminuendo nuovamente a 32.42 per $F = 32$.

F	d	Percentuale	MSE
8	5	23.44%	30.55
16	11	25.78%	32.47
24	17	26.56%	164.92
32	22	24.71%	32.42

Tabella 4: MSE al variare del blocco F , con tasso di taglio costante al 25%.

Questo comportamento evidenzia il legame tra la dimensionalità del blocco F e la periodicità del segnale geometrico quando si applica il taglio diretto della matrice:

- **caso per $F = 24$ ($\text{MSE} = 164.92$):** Poiché la dimensione originale dell'immagine (160) non è divisibile per 24, l'operazione di taglio rimuove gli ultimi 16 pixel della matrice ($160 - 24 \times 6 = 16$). Questo troncamento asimmetrico spezza a metà l'ultima fila di scacchi geometrici. La sottomatrice così tagliata introduce nello spettro DCT una forte componente a gradino artificiale che, privata del 75% delle alte frequenze a causa della soglia $d = 17$, genera una distorsione locale che distrugge la linearità dei bordi e fa aumentare il MSE dell'intera figura.
- **caso per $F = 8, 16, 32$ ($\text{MSE} \approx 32$):** Al contrario delle immagini naturali, l'errore non cresce all'aumentare del blocco se questo rimane un sottomultiplo esatto del piano immagine ($160/32 = 5$ blocchi perfetti). Quando F si allinea con la griglia geometrica, ogni sottomatrice cattura una porzione perfettamente simmetrica e speculare dei quadrati bianchi e neri. Lo scostamento spettrale dovuto al taglio frequenziale si distribuisce così in modo identico e bilanciato su tutti i blocchi, mantenendo l'errore costante e visivamente confinato alle sole linee di transizione netta dei bordi.